Control de Acceso y Protección de Rutas

Ionic Framework • Angular Standalone • Functional Guards (CanActivateFn) • Web Storage API

Módulo: Desarrollo de Interfaces (0488)
Ciclo: 2º Curso DAM • 2026-2027
Alumno: Daniel Daroca

Repositorio GitHub: `https://github.com/danidaroca/ionic-auth-guard`

Despliegue Vercel: `https://ionic-auth-guard.vercel.app`

CE 1.b: MVVM & Separación
Inyección desacoplada con `inject()`, reactividad pura con Signals y estado persistente.

CE 1.c: Enrutamiento Modular
Navegación jerárquica con *Lazy Loading* (`loadComponent: () => import(...)`).

CE 1.d: Functional Guards
Mecanismos `CanActivateFn` y factoría `roleGuard` devolviendo `UrlTree`.

CE 1.e: Persistencia Segura
Sincronización síncrona en `sessionStorage` con clave `DAM2_AUTH_SESSION`.

1. Fundamentos Técnicos: Del Enfoque Clásico a Functional Guards

Arquitectura Angular v16+

Evolución y Ventajas de CanActivateFn:

A partir de Angular 16+, el framework abandonó las clases con la interfaz `CanActivate` y constructores jerárquicos. El estándar actual son las **Functional Guards**:

- **Funciones Puras:** Se ejecutan en el pipeline antes de instanciar el componente destino.
- **Inyección con `inject()`:** Contexto de inyección directo y seguro, sin acoplar dependencias a clases ni requerir `providers` en módulos.
- **Redirección Atómica con `UrlTree`:** En lugar de retornar un booleano `false` y llamar manualmente a `router.navigate()` (lo que genera condiciones de carrera y parpadeos visuales), retornar `router.createUrlTree(['/login'])` detiene la navegación solicitada y conmuta a la ruta de forma atómica en el mismo ciclo.

Ciclo de Vida y Persistencia de Sesión:

Cuando el usuario refresca el navegador (F5) o escribe directamente una URL, el contexto en memoria de JavaScript se destruye y reinicia por completo.

`1. Usuario recarga /admin ➔ 2. Router activa guards ➔ 3. AuthService hidrata desde sessionStorage('DAM2_AUTH_SESSION') ➔ 4. Signal reactivo disponible ➔ 5. authGuard valida y permite o devuelve UrlTree('/login').`

El uso de `sessionStorage` garantiza que los datos persistan durante la navegación activa en la pestaña, pero se limpien automáticamente al cerrarla, asegurando máxima higiene de sesión.

2. Diagrama de Flujo y Pipeline de Navegación

Mecanismo de Intercepción



3. Control de Acceso Granular Basado en Roles (Propuesta del Ejercicio)

Requisito Sección 4

Se ha dado cumplimiento integral a los requerimientos propuestos en el punto 4 de la guía:

1. Perfil Admin: `admin / 123` (Rol: `'admin'`). Acceso total al panel `/admin`.

2. Perfil Operador: `operador / 123` (Rol: `'user'`). Acceso concedido a `/user-dashboard`.

3. Factoría roleGuard: Genera guards parametrizados `roleGuard(role)` que deniegan accesos no autorizados.

Código Relevante: Core de Seguridad y Servicios

AuthService (Signals + Storage), Functional Guards y Enrutamiento Modular

Archivo: auth.service.ts / auth.guard.ts
Patrón: Functional RBAC & Signals
Alumno: Daniel Daroca

1. Servicio de Autenticación con Angular Signals y Persistencia (src/app/core/services/auth.service.ts)

Multi-Rol & Signals

```typescript
import { Injectable, signal, computed } from '@angular/core';

export interface UserSession {
  username: string;
  role: 'admin' | 'user';
  isAuthenticated: boolean;
}

@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly STORAGE_KEY = 'DAM2_AUTH_SESSION';

  // Estado reactivo reactivado síncronamente desde sessionStorage al arrancar
  private _currentUser = signal<UserSession | null>(this.readSessionFromStorage());

  // Selectores computados expuestos a la aplicación
  public currentUser = this._currentUser.asReadonly();
  public isAuthenticated = computed(() => this._currentUser()?.isAuthenticated ?? false);

  /** Valida credenciales requeridas: admin/123 y operador/123 (Requisito Sección 4) */
  public login(user: string, pass: string): boolean {
    let sessionData: UserSession | null = null;

    if (user === 'admin' && pass === '123') {
      sessionData = { username: user, role: 'admin', isAuthenticated: true };
    } else if (user === 'operador' && pass === '123') {
      sessionData = { username: user, role: 'user', isAuthenticated: true };
    }

    if (sessionData) {
      // 1. Actualizar memoria reactiva
      this._currentUser.set(sessionData);
      // 2. Persistir en el navegador (Web Storage API)
      sessionStorage.setItem(this.STORAGE_KEY, JSON.stringify(sessionData));
      return true;
    }
    return false;
  }

  public logout(): void {
    sessionStorage.removeItem(this.STORAGE_KEY);
    this._currentUser.set(null);
  }

  private readSessionFromStorage(): UserSession | null {
    const raw = sessionStorage.getItem(this.STORAGE_KEY);
    if (!raw) return null;
    try { return JSON.parse(raw) as UserSession; }
    catch { sessionStorage.removeItem(this.STORAGE_KEY); return null; }
  }
}
```

2. auth.guard.ts (Functional Guard)

CanActivateFn

```typescript
import { inject } from '@angular/core';
import { CanActivateFn, Router, UrlTree } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state): boolean | UrlTree => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isAuthenticated()) {
    return true;
  }
  // Redirección atómica reteniendo la URL solicitada
  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url }
  });
};
```

3. role.guard.ts (Factoría RBAC)

Propuesta 4.2

```typescript
import { inject } from '@angular/core';
import { CanActivateFn, Router, UrlTree } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const roleGuard = (allowedRole: 'admin' | 'user' | string): CanActivateFn => {
  return (route, state): boolean | UrlTree => {
    const auth = inject(AuthService);
    const router = inject(Router);
    const session = auth.currentUser();

    if (!auth.isAuthenticated() || !session) {
      return router.createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
    }
    if (session.role === allowedRole) {
      return true;
    }
    // Si rol 'user' intenta entrar en /admin ➔ redirige a /user-dashboard
    if (session.role === 'user') {
      return router.createUrlTree(['/user-dashboard']);
    }
    return router.createUrlTree(['/admin']);
  };
};
```

4. Configuración de Rutas con Lazy Loading y Guards (src/app/app.routes.ts)

CE 1.c & CE 1.d

```typescript
import { Routes } from '@angular/router';
import { authGuard } from './core/guards/auth.guard';
import { roleGuard } from './core/guards/role.guard';

export const routes: Routes = [
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: 'login', loadComponent: () => import('./pages/login/login.page').then(m => m.LoginPage) },
  {
    path: 'admin',
    canActivate: [authGuard, roleGuard('admin')], // Protegido por ambos guards funcionales
    loadComponent: () => import('./pages/admin/admin.page').then(m => m.AdminPage)
```

Código Relevante: Componentes y Control Flow

Vistas Standalone: LoginPage, AdminPage y UserDashboardPage con Ionic

Vistas: Login, Admin, User-Dashboard
Sintaxis: Control Flow (@if), Signals
Alumno: Daniel Daroca

1. login.page.ts

TypeScript Standalone

```typescript
@Component({
  selector: 'app-login',
  templateUrl: './login.page.html',
  styleUrls: ['./login.page.scss'],
  standalone: true,
  imports: [CommonModule, FormsModule, IonContent, IonHeader, ...]
})
export class LoginPage implements OnInit {
  private auth = inject(AuthService);
  private router = inject(Router);
  private route = inject(ActivatedRoute);

  public username = '';
  public password = '';
  public errorMessage = '';

  ngOnInit(): void {
    if (this.auth.isAuthenticated()) {
      const role = this.auth.currentUser()?.role;
      this.router.navigate([role === 'admin' ? '/admin' : '/user-dashboard']);
    }
  }

  public handleLogin(): void {
    const ok = this.auth.login(this.username, this.password);
    if (ok) {
      const s = this.auth.currentUser();
      const ret = this.route.snapshot.queryParams['returnUrl'];
      if (ret && !(ret.startsWith('/admin') && s?.role === 'user')) {
        this.router.navigateByUrl(ret);
      } else {
        this.router.navigate([s?.role === 'admin' ? '/admin' : '/user-dashboard']);
      }
    } else {
      this.errorMessage = 'Credenciales no autorizadas. Use admin / 123 u operador / 123';
    }
  }
}
```

2. login.page.html

Ionic UI & Control Flow

```html
<ion-header [translucent]="true">
  <ion-toolbar color="primary">
    <ion-title>Control de Acceso</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-card class="login-card">
    <ion-card-header>
      <ion-card-title class="ion-text-center">Iniciar Sesión</ion-card-title>
      <ion-card-subtitle class="ion-text-center">Introduce credenciales</ion-card-subtitle>
    </ion-card-header>

    <ion-card-content>
      <ion-item fill="outline" class="ion-margin-bottom">
        <ion-label position="floating">Usuario</ion-label>
        <ion-input type="text" [(ngModel)]="username"></ion-input>
      </ion-item>

      <ion-item fill="outline" class="ion-margin-bottom">
        <ion-label position="floating">Contraseña</ion-label>
        <ion-input type="password" [(ngModel)]="password"></ion-input>
      </ion-item>

      <ion-button expand="block" (click)="handleLogin()">Acceder al Panel</ion-button>

      @if (errorMessage) {
        <ion-text color="danger">
          <p class="ion-text-center error-box">{{ errorMessage }}</p>
        </ion-text>
      }
    </ion-card-content>
  </ion-card>
</ion-content>
```

3. admin.page.ts & admin.page.html

Panel Superadmin

```typescript
// admin.page.ts
@Component({
  selector: 'app-admin',
  templateUrl: './admin.page.html',
  standalone: true,
  imports: [CommonModule, IonContent, IonHeader, ...]
})
export class AdminPage {
  private auth = inject(AuthService);
  private router = inject(Router);
  public currentSession = this.auth.currentUser;

  public handleLogout(): void {
    this.auth.logout();
    this.router.navigate(['/login']);
  }
}
```

```html
<!-- admin.page.html (fragmento) -->
<ion-card color="light">
  <ion-card-title>Bienvenido, {{ currentSession()?.username }}</ion-card-title>
  <ion-card-subtitle>Rol: {{ currentSession()?.role }}</ion-card-subtitle>
  <p>Superada validación de <code>authGuard</code> y <code>roleGuard('admin')</code>.</p>
</ion-card>
```

4. user-dashboard.page.ts & html

Panel Operador & Intercepción

```typescript
// user-dashboard.page.ts
@Component({
  selector: 'app-user-dashboard',
  templateUrl: './user-dashboard.page.html',
  standalone: true,
  imports: [CommonModule, IonContent, IonHeader, ...]
})
export class UserDashboardPage {
  private auth = inject(AuthService);
  private router = inject(Router);
  public currentSession = this.auth.currentUser;
  public feedbackMessage = '';

  /** Prueba de acceso directo requerida en Sección 4.2 */
  public async testDirectAdminAccess(): Promise<void> {
    await this.router.navigate(['/admin']);
    this.feedbackMessage = '⛔ [ACCESO DENEGADO]: roleGuard("admin") bloqueó el acceso.';
  }
}
```

Evidencias Visuales, Verificación y Pruebas Unitarias

Capturas de Ejecución de UI, DevTools SessionStorage y Suite Vitest

Testing: 16/16 Tests Superados
Navegador: Chromium DevTools
Alumno: Daniel Daroca

1. Simulación Visual de Pantallas y Comportamiento de Seguridad

Captura de Interfaz

http://localhost:8100/login

Control de Acceso Ionic 9

Iniciar Sesión

Introduce tus credenciales

Usuario: **operador**

Pass: •••

Acceder al Panel

http://localhost:8100/admin

Zona Segura: Panel de Control Cerrar Sesión

Rol: admin • Sesión Activa

Bienvenido, admin

✅ Has superado la validación del enrutador (`authGuard` y `roleGuard('admin')`).

http://localhost:8100/user-dashboard

Panel Operador: Zona de Usuario Cerrar Sesión

Rol: user **Bienvenido, operador**

Verificación Requisito 4.2 (Intento a /admin):

Al escribir directamente `/admin` o pulsar el botón de prueba, `roleGuard('admin')` intercepta y redirige aquí.

⛔ **ACCESO DENEGADO POR ROLEGUARD: Redirigido a /user-dashboard.**

DevTools > Application > Session Storage > http://localhost:8100

```
Key: DAM2_AUTH_SESSION
Value: {
  "username": "admin",
  "role": "admin",
  "isAuthenticated": true
}
// Sincronizado automáticamente mediante Web Storage API
```

2. Matriz de Pruebas y Casos de Verificación Técnica

Resultados de Pruebas

Caso de Prueba	Estado Previo	Acción Realizada	Respuesta del Guard / Router	Resultado
1. Acceso Anónimo a /admin	Sin sesión	URL directa `http://localhost:8100/admin`	`authGuard` cancela y devuelve `UrlTree('/login?returnUrl=/admin')`	✓ SUPERADO
2. Login Admin Válido	En `/login`	Introduce `admin / 123`	`AuthService` almacena clave en `sessionStorage` y navega a `/admin`	✓ SUPERADO
3. Persistencia de Sesión (F5)	En `/admin` autenticado	Refrescar navegador (F5 / recarga total)	`readSessionFromStorage()` rehidrata Signal en milisegundos; no se desloguea	✓ SUPERADO
4. Login Operador Válido	En `/login`	Introduce `operador / 123`	Asigna rol `'user'` y redirige automáticamente a `/user-dashboard`	✓ SUPERADO
5. Ataque Inter-Rol a /admin	Sesión como `'user'`	Escribe manualmente en barra `/admin`	`roleGuard('admin')` deniega acceso y genera `UrlTree('/user-dashboard')`	✓ SUPERADO
6. Cierre de Sesión Seguro	En cualquier panel	Pulsa "Cerrar Sesión"	Elimina clave `DAM2_AUTH_SESSION`, Signal a `null` y redirige a `/login`	✓ SUPERADO

3. Verificación Automatizada con Vitest (16 de 16 Tests Pasados)

CI / Testing

✓ 7 Test Files pasados (100%) • ✓ 16 Tests unitarios superados en 13.09s

- `auth.service.spec.ts (6 tests: create, default null, login admin, login user, reject invalid, logout)`
- `role.guard.spec.ts (4 tests: redirect unauth, allow admin, deny user to admin, allow user to dashboard)`
- `auth.guard.spec.ts (2 tests: redirect unauth with returnUrl, allow auth)`
- `admin.page.spec.ts (1 test) • user-dashboard.page.spec.ts (1 test) • login.page.spec.ts (1 test) • app.component.spec.ts (1 test)`